SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

LintasNiaga

Choose the best-value supplier with less hidden risk

Core System Claim

LintasNiaga is not a generic chatbot. It is a FastAPI-owned procurement decision system where supplier quote extraction, snapshot-grounded market context, deterministic landed-cost simulation, and bounded GLM reasoning work together to produce a traceable supplier, timing, and hedge recommendation.

Field	Details
Document Type	System Analysis Documentation
Version	1.0 Submission Draft
Repo Grounding	Latest repo review: current SAD, architecture blueprint, tech stack blueprint, PRD, backend routes, quote ingest, AI orchestrator, and analysis services
Prepared For	UMHackathon 2026 Preliminary Round
Prepared By	404 Not Founders Team
Date	2026-04-25

Table of Contents

No.	Contents	Pg.
1	Introduction & Purpose	3
2	Background & Architecture Evolution	3
3	Target Stakeholders	4
4	System Architecture & Design	5
5	GLM As Service Layer	6
6	Key Data Flows & Logical Data Model	8
7	Functional Requirements & Scope	10
8	Non-Functional Requirements	10
9	Monitoring, Evaluation, Assumptions & Dependencies	11
10	Project Management & Team Contributions	12
11	Recommendations, Risks & Open Questions	13

1. Introduction & Purpose

1.1 Introduction

LintasNiaga is a procurement decision-support system for Malaysian plastics SMEs importing polypropylene (PP) resin. It processes supplier quotation PDFs, refreshes relevant market and logistics context, computes landed-cost scenarios, and assembles an explainable recommendation on which supplier to choose, whether to lock now or wait, and how much FX exposure to hedge.

The system is intentionally narrow. It focuses on a real procurement decision instead of trying to become a general sourcing platform. The current architecture is designed to provide decision intelligence where the backend owns the calculations, the model reasons over structured evidence, and the user receives an action-ready decision with traceability.

1.2 Purpose of this SAD

This System Analysis Documentation explains the technical scope, architecture, data flow, GLM integration, key system dependencies, monitoring assumptions, and project constraints behind LintasNiaga. It is written for UMHackathon judges, developers, testers, and mentors who need to understand how the system works from upload to recommendation.

1.3 Key Elements Covered

Element	Coverage in this SAD
Architecture	Frontend, backend, provider adapters, repositories, local file-backed data, in-memory run state, and GLM/Langfuse traceability.
Data Flows	How PDFs, extracted quote fields, reference data, snapshots, AI context, and recommendation payloads move through the system.
Model Process	How GLM is called for quote extraction fallback, bounded recommendation reasoning, streamed explanation, and bank-instruction wording.
Role of Reference	This document acts as the shared reference for implementation, testing, demo explanation, and future productionisation.

2. Background & Architectural Evolution

2.1 Background of the Problem

SME procurement teams often compare raw-material supplier quotations through spreadsheets, email attachments, and intuition. For PP resin importers, the supplier unit price is only the visible part of the decision. True landed cost is affected by the quote currency, freight, tariffs, MOQ, lead time, supplier reliability, FX movements, oil-linked logistics pressures, weather disruptions, holidays, and market benchmark changes.

The system therefore addresses a decision problem, not only a document extraction problem. LintasNiaga turns fragmented quote documents and external market context into a structured, defensible procurement recommendation.

2.2 Previous Version and Current Version

Area	Earlier Direction	Current Updated Repo Reality
Architecture	Ingestion-aligned backend concept.	Backend-owned procurement analysis system with real quote workflow, snapshot refresh, Monte Carlo, GLM reasoning, traceability, hedge replay, and bank PDF flow.
AI Role	Broad AI module / generic analysis layer.	Thin LangGraph reasoning flow; deterministic backend services still own business logic.
Data Layer	SQLite/Supabase is discussed as a possible persistence.	Current runtime uses local JSON snapshots, raw artifacts, uploaded PDFs, and in-memory quote/run state. Supabase remains on the roadmap.
Simulation	Earlier risk-driver approximation.	FX + Brent correlated Monte Carlo through the current simulation service, with stable hedge replay.
Resin Data	Potential commodity input.	SunSirs PP benchmark is implemented as quote-vs-market evidence, not a stochastic Monte Carlo driver.

2.3 Changes in Major Architectural Components

- Added scheduled background refresh loops for news, holidays, resin, market, weather, and macro context.
- Added stricter analysis, refresh and validation before computing results.
- Added Langfuse-backed traceability for GLM calls and simulation spans.
- Added hedge replay endpoint and bank-instruction draft endpoint.
- Added richer result payloads: p10/p50/p90 fan chart, risk drivers, resin benchmark, market price risk labels, news events, and trace URLs.
- Kept roadmap-only features out of the active architecture: no direct WorldFirst execution, no active Supabase integration, no queue workers, no enterprise auth layer.

3. Target Stakeholders

Stakeholder	Role	Expectations
SME Procurement User	Uploads supplier quotes, reviews extracted fields, runs analysis, and interprets recommendations.	Fast quote comparison, clear landed-cost view, understandable reasons, hedge guidance, no fake certainty.
SME Owner / Finance Lead	Approves purchase and hedge decision.	Defensible supplier choice, risk visibility, margin protection, and traceable explanation.
Development Team	Builds and maintains frontend, backend, ingestion, simulation, and AI orchestration.	Clear API contracts, modular services, grounded architecture, and manageable scope.
QA Team / Testers	Validates backend behavior, data contracts, frontend flows, and failure paths.	Defined route surface, expected payload shape, manual test matrix, and traceability checks.
Hackathon Judges / Mentors	Evaluate technical depth, sponsor fit, GLM centrality, and business impact.	A decision intelligence product, not a hardcoded dashboard or generic chatbot.

4. System Architecture & Design

4.1 High-Level Architecture Overview

Type	Details
System Type	Web-based procurement decision intelligence system
Frontend	Next.js 16 + React 19 application in apps/web
Backend	FastAPI application in apps/api
Architecture Style	Service-oriented monolith: modular backend services inside one FastAPI app
Data Layer	Local reference JSON, snapshots, raw artifacts, uploaded PDFs, and in-memory runtime state
AI Layer	GLM-compatible model through provider wrapper, thin LangGraph flow, Langfuse traceability
Current Deployment Reality	Local/dev-oriented stack; hosted Vercel/Render/Supabase remain roadmap unless separately configured

Architecture Principle

Frontend is thin. FastAPI owns the workflow. Deterministic cost/risk math runs before GLM reasoning. External data enters analysis through normalized snapshots. AI explains and reasons over backend results; it does not replace the cost engine.

Simplified Architecture Diagram

```
[SME Procurement User]
        |
        v
[Next.js Frontend]
Upload -> Review/Repair -> Progress -> Results Dashboard
        |
        | REST + SSE
        v
[FastAPI Backend Decision Engine]
Quote extraction + validation
Reference/snapshot loading
Landed-cost + Monte Carlo risk engine
Ranking + hedge replay + bank draft
        |
        +----> [GLM Reasoning Layer]
        |      Thin LangGraph node + strict JSON output
        |      Langfuse traceability
        |
        +----> [Data and Market Context]
               Uploaded PDFs
               data/reference: freight, tariff, supplier seeds
               data/snapshots: FX, oil, weather, macro, news, resin, holidays
               data/raw: source evidence
```

4.2 Major Backend Components

Component	Technical Role
Quote Ingest Service	Saves PDF, extracts text using PyMuPDF, falls back to GLM vision extraction, and stores in-memory QuoteState.
Quote Validation Service	Enforces scope: FOB incoterm, supported currencies, recognized origin, and required fields.
Analysis Run Service	Main orchestrator: loads reference data, refreshes snapshots, computes costs, builds AI context, stores result.
Cost / Ranking Services	Compute material, freight, tariff, MOQ, trust penalty and rank supplier quotes.
FX + Brent Monte Carlo	Simulates p10/p50/p90 landed-cost bands using FX and oil snapshots, stable seeds, and hedge ratio.
AI Orchestrator	Thin LangGraph flow that sends structured backend context to GLM and parses strict JSON.
Recommendation Assembler	Applies guardrails so AI cannot freely override deterministic ranking.
Snapshot / Raw / Reference Repositories	Read/write local JSON snapshots, raw artifacts, and static reference anchors.

5. GLM As Service Layer

The GLM integration is implemented as a service layer, not a vague “AI module”. Backend services prepare structured input, call the GLM provider wrapper, parse the response, enforce strict JSON where needed, and pass only validated output to downstream components.

5.1 GLM Dependency Diagram

GLM Service Dependency Flow

```
Quote PDF or Analysis Context
        |
        v
FastAPI Service prepares bounded prompt
        |
        v
GLM Provider Wrapper
- model API key/base URL/model name
- temperature = 0
- Langfuse callbacks
- strict JSON parsing for extraction/recommendation
        |
        +----> GLM API
        |
        +----> Langfuse Trace
        |
        v
Parsed Response
- Extracted quote fields, or
- Recommendation JSON, or
- Streamed analyst explanation, or
- Bank instruction wording
        |
        v
Backend validation / assembler / API response
```

5.2 Prompt Construction and Context Window

GLM Use Case	Prompt / Context Input	Response Handling
Quote extraction fallback	System instruction + supplier quotation image + field extraction prompt.	Strict JSON parsed into ExtractedQuote; trace URL/ID stored when available.
Recommendation reasoning	Ranked quotes, landed-cost outputs, FX/oil simulation, resin benchmark, news, macro, weather, holiday, tariff, freight context.	Strict JSON parsed into recommendation fields and passed to Recommendation Assembler.
Streamed analyst explanation	Same structured analysis context, but prompt asks for user-friendly explanation.	SSE chunks streamed to frontend; stream trace URL captured when available.
Bank instruction wording	Selected quote, hedge scenario, business rationale, fallback draft fields.	Structured draft returned to frontend PDF renderer.

5.3 Token and Input Handling

- PDF quote extraction is scoped to the first two pages to control image and prompt size.
- The analysis prompt is built from structured backend summaries, not raw full HTML or full PDFs.
- News/context is limited to the top relevant events rather than unlimited articles.
- The model max token setting is configured at the provider level; the backend expects strict JSON for core decision calls.
- Oversized or unusable quote inputs are handled through validation/repair rather than silently sent as unbounded context.

5.4 Fallback and Failure Behavior

Failure Case	System Behavior
Quote extraction incomplete	User repair workflow allows correction before analysis.
Unsupported quote scope	Quote is excluded or marked invalid; analysis only uses valid quotes.
Critical snapshot missing/stale	Strict analysis can block and return an explainable error instead of inventing a result.
GLM recommendation output invalid	Analysis fails explicitly because bounded JSON recommendation is required.
Bank instruction GLM failure	Backend has deterministic fallback draft path for demo continuity.
Hedge simulation full replay unavailable	Endpoint has a simpler mathematical fallback if stored scenario inputs are unavailable.

6. Key Data Flows & Logical Data Model

6.1 Main User Interaction Sequence

Sequence Diagram (Textual)

```
1. User uploads supplier quote PDFs.
2. Frontend sends PDFs to POST /quotes/upload.
3. Backend saves PDFs and extracts fields.
4. User reviews and repairs extracted fields.
5. Frontend calls POST /analysis/run with quote IDs, quantity, urgency, hedge preference.
6. Backend refreshes required snapshots and validates reference data.
7. Backend computes landed-cost and p10/p50/p90 risk bands.
8. Backend sends structured context to GLM through thin LangGraph flow.
9. GLM returns strict JSON recommendation; Langfuse stores trace.
10. Backend assembles canonical result and stores run state in memory.
11. Frontend renders dashboard, fan chart, risk drivers, hedge replay, and bank PDF action.
```

6.2 Data Flow Diagram

DFD Level 0/1 (Textual)

```
External Entity: SME User
        |
        | Supplier PDFs + quantity + urgency + hedge preference
        v
Process 1: Quote Intake and Repair
        | ExtractedQuote + validation status
        v
Data Store: In-memory QuoteState + uploaded PDF files
        |
        v
Process 2: Analysis Run Service
        | uses
        +--> Data Store: data/reference (freight, tariff, supplier seeds)
        +--> Data Store: data/snapshots (FX, oil, weather, macro, news, resin, holidays)
        +--> Data Store: data/raw (source evidence where needed)
        |
        v
Process 3: Cost + Risk + Monte Carlo Engine
        | ranked quotes + scenarios
        v
Process 4: GLM Reasoning Service
        | strict JSON + Langfuse trace
        v
Process 5: Recommendation Assembler
        |
        v
External Entity: SME User receives dashboard, hedge replay, bank PDF draft
```

6.3 Logical Data Model

The current system is not database-backed in production form. It uses local file-backed storage and in-memory runtime dictionaries. The table below represents the logical data model that the current implementation follows and can later be migrated into SQLite/Postgres/Supabase.

Logical Entity	Current Storage	Key Fields / Meaning
QuoteUpload	Uploaded file + QuoteState memory	upload_id, filename, storage_path, uploaded_at, status
ExtractedQuote	QuoteState memory	quote_id, supplier, origin, incoterm, unit_price, currency, MOQ, lead_time, payment_terms
QuoteValidationResult	QuoteState memory	quote_id, status, reason_codes, missing_fields
ReferenceData	data/reference/*.json	freight_rates, tariff rules, ports, supplier seeds
SnapshotEnvelope	data/snapshots/*	dataset, source, status, as_of, record_count, data
RawArtifact	data/raw/*	raw HTML/text/source evidence for debugging and provenance
AnalysisResultPayload	In-memory run result	run_id, recommendation, ranked_quotes, scenarios, risk drivers, trace URLs
HedgeScenarioResult	Derived on request / in memory	hedge ratio, p10/p50/p90 envelopes, impact vs unhedged
BankInstructionDraft	Generated response	supplier, currency, amount, tenor, hedge ratio, justification, risk rationale

7. Functional Requirements & Scope

7.1 Minimum Viable Product

#	Feature	Description
1	Quote Upload and Extraction	User uploads up to 5 supplier PDFs. Backend saves files, extracts fields, and falls back to GLM vision extraction when text parsing is insufficient.
2	Review and Repair Workflow	User checks extracted supplier, origin, incoterm, currency, unit price, MOQ, and lead time before analysis.
3	Landed-Cost Comparison	Backend computes material, freight, tariff, MOQ, trust penalty, and ranked quote results.
4	FX + Brent Monte Carlo Risk Bands	System generates p10/p50/p90 landed-cost fan chart and supports hedge replay with stable seeds.
5	Traceable GLM Recommendation	GLM reasons over structured backend results and returns supplier, timing, hedge ratio, reasons, caveat, and why-not explanation.

7.2 Features Included

- PDF quote upload, extraction, validation, and user repair.
- Comparison mode is used when at least two valid quotes exist.
- Single-quote evaluation mode when only one valid quote remains.
- External snapshot refresh for FX, Brent, weather, macro, news, holidays, and PP resin benchmark context.
- SunSirs PP resin quote vs. market fairness analysis.
- Landed-cost scenario outputs and risk-driver breakdown.
- Hedge replay and bank-instruction draft generation.
- Langfuse traceability through health and analysis trace endpoints.

7.3 Out of Scope / Future Enhancements

- Direct WorldFirst or bank execution integration.
- Enterprise authentication and role-based access control.
- Durable database-backed workflow state.
- Multi-industry procurement beyond PP resin.
- Full supplier relationship management and contract lifecycle management.
- Queue-based ingestion workers, Redis/Celery, and production CI/CD pipeline.
- Hosted Supabase/Postgres storage unless separately implemented after this submission.

8. Non-Functional Requirements

Quality	Requirement	Implementation / Current Status
Reliability	System should not silently invent procurement results when critical data is missing.	Strict refresh and validation can block analysis with explainable error. Some fallback behavior remains for non-critical/demo paths.
Traceability	GLM calls must be visible for judging and debugging.	Langfuse callbacks, /health/langfuse, /analysis/{run_id}/traceability, trace_url and stream_trace_url in payload.
Maintainability	Code should be modular enough for quick hackathon iteration.	Provider/service/repository separation, Pydantic schemas, route groups, local data contracts.
Performance	User should see progress and avoid dead UI while analysis runs.	SSE analyst stream and result polling/progress page. Live refresh may still add latency.
Cost Efficiency	Avoid sending raw oversized context to GLM.	Structured summaries, top event selection, first-two-page PDF extraction, strict JSON output, deterministic math outside model.
Security/Integrity	Prevent unsupported data from entering decisions.	Validation gates for FOB, currency, origin, and required quote fields. No enterprise auth yet.
Scalability	Hackathon prototype should be extensible to persistent production architecture.	Current state is memory/file-backed; roadmap includes database-backed run state and hosted persistence.

9. Monitoring, Evaluation, Assumptions and Dependencies

9.1 Technical Evaluation

- Backend automated testing is managed through pytest.
- Current QA documentation records backend tests for analysis routes, cost engine, FX simulation, market data, quote ingestion, validation, ranking, reference data, risk-driver services, and SunSirs resin parsing.
- Frontend test tooling exists, but manual end-to-end workflow validation remains important for upload, review, result rendering, hedge replay, PDF generation, and traceability visibility.

9.2 Rollout and Demo Strategy

Stage	Strategy
Local Golden Demo	Use a known-good environment with configured API keys, prepared reference files, and verified snapshots.
Frozen Snapshot Mode	Use saved snapshots if live providers are unstable close to presentation time.
Manual E2E Smoke Test	Before submission, run upload -> review -> analysis -> result -> hedge replay -> bank PDF -> traceability.
Production Roadmap	Move active run state to persistent DB and separate refresh jobs if the product is scaled beyond hackathon.

9.3 Emergency Rollback and Golden Release

- Keep a known-good local branch or tagged commit for demo delivery.
- Keep curated reference files and latest valid snapshots available to avoid live upstream failures during judging.
- If a provider fails, show transparent failure or switch to prepared snapshot mode instead of hardcoding fake results.
- If GLM/Langfuse credentials fail, the demo should clearly state the dependency issue; traceability is part of the judged AI claim.

9.4 Priority Matrix

Priority	Trigger Condition	Action
P1 Critical	No valid quotes, GLM unavailable, Langfuse trace not captured for recommendation, or critical FX/oil snapshot missing.	Block analysis or switch to controlled demo snapshot; show explainable error.
P2 High	Weather/news/resin refresh fails, but the last valid context exists.	Use the latest valid snapshot if policy allows; label freshness and avoid overclaiming.
P3 Medium	Streamed analyst explanation fails after the main recommendation succeeds.	Keep the result dashboard available; show non-blocking stream error.
P4 Low	Visual/UI issue not affecting the recommendation payload.	Document and fix after the core demo path is stable.

9.5 Monitoring Plan

Monitor Area	Signal	Action
API Health	/health and backend logs	Confirm FastAPI is running and route groups are loaded.
Langfuse	/health/langfuse and trace URLs	Confirm GLM calls are traceable.
Snapshots	/snapshots/latest/* and ingestion logs	Check FX, oil, weather, macro, news, resin, and holiday freshness.
Analysis Runs	Backend errors and result payload completeness	Detect broken analysis pipeline or missing payload fields.
Frontend Flow	Manual smoke test	Verify upload, review, dashboard, hedge slider, PDF, trace link.

9.6 Assumptions

- Users can provide readable supplier quote PDFs.
- The hackathon demo focuses on PP resin imports into Malaysia, not every procurement category.
- Supported quote scope is FOB with supported currencies and origins.
- External providers can be accessed during development, or valid snapshots can be prepared before the demo.
- Local file-backed storage and in-memory run state are acceptable for the hackathon MVP.
- The GLM endpoint and Langfuse credentials are configured in the demo environment.

9.7 External Dependencies

Tool / Source	Purpose	Risk
Z.AI / GLM-compatible model endpoint	Quote extraction fallback, recommendation reasoning, streamed explanation, bank wording.	High: core AI reasoning depends on it. Mitigation: strict errors, deterministic math stays outside model, repair flow.
Langfuse	AI traceability and judge/debug visibility.	High for traceability claim; configure before demo.
yfinance	FX and Brent historical snapshots for simulation.	Medium: provider instability can block strict analysis. Prepare snapshots.
OpenWeather	Port weather risk and delay signals.	Medium: API/key limits. Use latest valid snapshot if acceptable.
OpenDOSM / data.gov.my	Malaysia macro/trade context.	Medium: refresh failure affects context.
GNews / RSS mode	Current event risk context.	Medium: news freshness and rate limits.
SunSirs	PP resin benchmark and quote-vs-market check.	Medium: scraping/source format may change. Raw evidence retained.
Local JSON data/reference	Freight/tariff/supplier seed anchors.	Medium: must be clearly labelled as curated/reference assumptions.

10. Project Management and Team Contributions

10.1 Development Timeline

Phase	Focus	Justification
Day 1-2	Problem framing, sponsor fit, wedge selection	Narrow scope to Malaysia PP resin procurement decision intelligence.
Day 3-4	PRD/SAD/architecture and data model	Define backend-owned workflow, data sources, and GLM role.
Day 5-6	Quote ingestion, validation, reference/snapshot layer	Make uploaded quotes analysis-ready and defensible.
Day 7-8	Cost engine, Monte Carlo, risk drivers, GLM reasoning	Build core decision capability and traceability.
Day 9	Frontend workflow, result dashboard, hedge replay, bank PDF	Make the system demoable and judge-readable.
Day 10	QA, docs, demo polish, fallback preparation	Stabilize happy path and document limitations honestly.

10.2 Team Member Roles

Member / Role	Responsibility
Product / Strategy Lead	Problem framing, mentor feedback synthesis, PRD/SAD alignment, demo narrative.
Backend Lead	FastAPI routes, quote ingestion, validation, analysis orchestration, provider/repository layers.
AI / Data Lead	GLM prompting, LangGraph orchestration, Langfuse tracing, SunSirs/market/snapshot ingestion.
Simulation / Analytics Lead	Landed-cost engine, FX + Brent Monte Carlo, hedge replay, risk-driver logic.
Frontend Lead	Next.js UI, upload/review/results dashboard, charts, SSE, bank PDF rendering.
QA / Demo Lead	Backend tests, manual E2E testing, fallback snapshots, and final demo flow.

11. Recommendations, Risks and Open Questions

11.1 Recommendations for Scalability, Performance and Reliability

- Move QuoteState and AnalysisResultPayload from memory into SQLite/Postgres/Supabase before any production pilot.
- Separate ingestion refresh jobs from request-time analysis once the system is hosted.
- Add a snapshot freeze/demo mode to reduce provider risk during live presentations.
- Make freight unit semantics explicit: mt, container_20ft, container_40ft, shipment_total.
- Add at least one frontend test for result rendering and hedge slider behavior.
- Standardize settings into one configuration layer to reduce env drift.

11.2 Key Risks

Risk	Impact	Mitigation
External provider instability	Analysis may fail or slow down.	Prepare valid snapshots and show clear failure states.
In-memory state	Runs disappear after backend restart.	Accept for hackathon; migrate to persistent store later.
GLM/Langfuse misconfiguration	Core AI traceability may fail.	Run /health/langfuse before demo; keep env checklist.
Freight/tariff assumptions	Incorrect landed-cost interpretation.	Label as curated references; add source notes and validation status.
Prompt/context drift	Model returns unusable JSON or overclaims.	Strict prompts, JSON parser, bounded assembler guardrails.

11.3 Final System Analysis Statement

LintasNiaga currently operates as a coherent hackathon-ready procurement decision system. Its strongest technical characteristic is the separation between deterministic backend economics and bounded AI reasoning: the backend validates quotes, loads snapshots, computes landed cost and risk bands, and ranks suppliers before the GLM explains the decision. This makes the solution credible for UMHackathon because the model is central to interpretation and explanation, but the financial math remains controlled, auditable, and traceable.